

FinTrack

Project	FinTrack — personal finance tracker (SPA frontend)
Document	Frontend Technical Requirements v1.0
Companion docs	Business Requirements v1.0, API Contract v1.0, Design (HTML reference)
Audience	Frontend developer (implementation) & automated code review
Purpose	Defines <i>how</i> the frontend is built — stack, architecture, and conventions. The basis for code-review rules.

How to read this document. The Business Requirements say *what* to build; the API Contract says *what data* flows; this document says *how* to build it well. It is intentionally high-level: it describes patterns and rules, not full implementations. Where code appears, it illustrates a pattern — it is not code to copy verbatim. The developer is expected to understand and apply each pattern themselves.

1. Technology Stack

The stack is chosen to reflect what a modern production React codebase looks like in 2026. Each tool has a defined role; do not substitute or duplicate roles.

Concern	Tool	Role / requirement
Build tool	Vite	Dev server + bundler. No Create React App.
Language	TypeScript	<code>strict: true</code> . Mandatory across the whole codebase.
UI library	React 18/19	Function components + hooks only.
Routing	React Router	Client-side routing, protected routes.
Server state	TanStack Query	All data from the API. Caching, refetch, mutations.
Client state	Zustand	Only UI/local state that isn't server data (e.g. auth token in memory, modal state).
Forms	React Hook Form + Zod	All forms. Validation schemas in Zod.
Styling	Tailwind CSS + shadcn/ui	Utility-first + accessible component primitives.
HTTP	Axios	Single configured instance with interceptors.
Charts	Recharts	Dashboard visualisations.
Testing	Vitest + RTL	Optional but encouraged; a few meaningful tests.
Quality	ESLint + Prettier	Enforced. Code must pass lint before merge.

Rule — one tool per concern. Server data lives in TanStack Query, never copied into Zustand. Zustand holds only what is genuinely client-side. Mixing these is the most common architectural mistake and will be flagged in review.

2. Design Tokens

Extracted from the approved design. These are set up **once** in Tailwind config / CSS variables and used everywhere by semantic name. Never hardcode a hex value in a component.

Palette

Token	Value	Meaning
primary (brand)	 #2563EB	Accent: buttons, links, active nav. Themeable.
income	 #059669	Income amounts & positive values. Fixed semantics.
expense	 #E11D48	Expense amounts & negative values. Fixed semantics.
foreground	 #0F172A	Primary text (slate-900).
muted-foreground	 #64748B	Secondary text (slate-500).
border	 #E9EDF3	Dividers, card borders.
muted / surface	 #F8FAFC	Subtle backgrounds.
background	 #FFFFFF	Page/card background.

Rule — separate brand from semantics. `primary` may change with theming. `income` / `expense` must NOT be reused for branding — they carry meaning (green = money in, red = money out). Keep them as distinct tokens.

Typography, radius, spacing

- Font:** Geist (UI), Geist Mono (numbers/amounts). Amounts render in the mono font for alignment.

- **Radius:** base ~9–13px (soft, shadcn-like). Use one or two radius tokens, not arbitrary values.
- **Spacing:** use Tailwind's spacing scale consistently; avoid magic pixel values.

On the design HTML file. The exported design is a *visual reference*, not source code. Open it in a browser and match it; do not copy its generated markup — it is compiled output from a design tool, not components written on this stack. Reproduce each screen with your own shadcn/Tailwind components.

3. Project Architecture

Use a **feature-based** structure: code is grouped by domain feature, not by file type. This keeps related logic together and scales far better than a flat `components/` + `hooks/` split.

```
src/
  app/           // app shell: router, providers, layout
  features/
    auth/        // login, register, session, guards
    transactions/ // list, form, filters, hooks, types
    accounts/
    categories/
    dashboard/   // analytics widgets, charts
  components/ui/ // shared shadcn primitives (button, input, dialog...)
  lib/           // api client, query client, helpers (money format, dates)
  types/         // shared/global types
```

Inside a feature folder, keep its own components, hooks (queries/mutations), Zod schemas, and types together. A feature owns its slice of the app end to end.

Rule — features are self-contained. A component in `dashboard/` should not reach into the internals of `transactions/`. Share through well-defined exports or `lib/`, not deep imports.

4. Server State (TanStack Query)

Every piece of data that comes from the API is server state and is owned by TanStack Query. Components never fetch directly; they call a query/mutation hook.

- **Queries** for reads (`useTransactions` , `useAccounts` , `useDashboardSummary`).
- **Mutations** for writes (`useCreateTransaction`), which invalidate the relevant queries on success so the UI stays fresh.
- Use consistent, structured **query keys** that include filters, e.g. `['transactions', filters]`, so filtered lists cache correctly.

PATTERN (ILLUSTRATIVE, NOT FULL CODE)

```
// a mutation invalidates what it changed
useMutation({
  mutationFn: createTransaction,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['transactions'] })
    queryClient.invalidateQueries({ queryKey: ['analytics'] })
  },
})
```

Rule. Loading, error, and empty states come from Query's own flags (`isLoading` , `isError` , empty `data`). Every list and widget handles all three — never render a blank screen.

Anti-pattern. Copying fetched data into `useState` or Zustand "to keep a local copy". This creates two sources of truth that drift. Read straight from the query.

5. Client State (Zustand)

Zustand holds only state that is **not** server data and needs to be shared across components:

- The in-memory access token + current user (the auth store).
- Global UI state that several components need (e.g. theme).

Local, single-component UI state (a dropdown open/closed) stays in `useState` — it does not belong in a global store.

6. Forms (React Hook Form + Zod)

All forms (login, register, transaction, account, category) follow the same pattern:

1. A **Zod schema** defines the shape and validation rules.
2. React Hook Form drives the form via `zodResolver(schema)`.
3. The inferred type (`z.infer<typeof schema>`) is the form's TypeScript type — one source of truth for shape + validation.
4. Validation errors render inline next to fields (per US requirements).

PATTERN (ILLUSTRATIVE)

```
const schema = z.object({
  amount: z.number().positive(),
  categoryId: z.string().min(1),
  date: z.string(),
})
type FormValues = z.infer<typeof schema>
```

Rule — validate on the client, but the server is the source of truth. Client validation is for UX. Always still handle server-side 422 errors from the API Contract and surface them on the form.

Rule — match business rules. The transaction form must filter categories by the selected type (expense form shows expense categories only) and prevent zero/negative amounts — matching US-D1 and the API Contract.

7. API Layer & Auth

A single configured Axios instance is the only way the app talks to the backend. No scattered `fetch` calls.

Requirements

- One `lib/api.ts` Axios instance with `baseURL` and `withCredentials: true` (so the refresh cookie travels).
- A **request interceptor** attaches `Authorization: Bearer <accessToken>` from the auth store.
- A **response interceptor** catches 401, calls `/auth/refresh` once, retries the original request; if refresh fails, it clears the session and redirects to login.
- On app start, attempt a silent refresh to restore the session (access token lives only in memory).
- All request/response types come from the **API Contract** — typed, no `any`.

Rule. Protected routes are guarded by a `<ProtectedRoute>` wrapper that redirects unauthenticated users to `/login` and returns them to their target after login (US-A5).

Anti-pattern. Storing the access token in `localStorage`. It lives in memory (auth store); the refresh token is an httpOnly cookie the frontend never touches. This matches the security model in the API Contract.

8. TypeScript Conventions

- `strict` mode on. No implicit `any`; no explicit `any` to silence the compiler.
- Domain types (Transaction, Account, Category) are defined once, derived from the API Contract shapes, and reused.
- Prefer inference where it's clear; annotate function boundaries and public hook return types.
- Money is an integer (minor units) in types — formatting to a display string is a dedicated helper in `lib/`.

Rule — money handling. Never do currency math with floats in components. Keep amounts as integer minor units; centralise parse/format in one helper. This mirrors the API Contract and prevents rounding bugs.

9. UX & Quality Bar

Area	Requirement
States	Every data view handles loading, empty, and error explicitly. No blank screens, no silent failures.
Feedback	Every action gives visible feedback (spinner, success, error) within ~1s. Mutations disable their trigger while pending.
Destructive actions	Delete always confirms first (dialog).
Optimistic updates	Encouraged for quick actions (e.g. delete) where it improves feel; must roll back on error.
Accessibility	Use shadcn/ui primitives (they're accessible by default). Label inputs; keyboard-usable dialogs.
Responsiveness	Usable on a laptop screen at minimum; graceful narrowing is a plus.
Consistency	Colours, spacing, and radii come from tokens only. No one-off hex values.

10. Workflow & Definition of Done

Git & review flow

- Work happens on feature branches; changes land via Pull Request — never direct pushes to main.
- Each PR is scoped to one feature or fix, with a clear description.
- Every PR triggers the automated code review (mentor bot) and must pass lint + typecheck.
- Commit messages are clear and conventional (e.g. `feat: add transaction filters`).

A feature is "done" when...

- It satisfies its user story's acceptance criteria (Business Requirements).
- It uses the correct data flow (Query for server, Zustand only where justified).
- Loading / empty / error states are handled.
- It is fully typed (no `any`) and passes lint + typecheck.
- It visually matches the design reference and uses design tokens.
- Forms validate with Zod and handle server errors.

Suggested build order

1. Project setup: Vite + TS + Tailwind + tokens + shadcn + router + query client.
2. API layer + auth (interceptors, store, ProtectedRoute) — unblocks everything.
3. Auth screens (login / register / session restore).
4. Categories & Accounts CRUD (simpler features, establish the patterns).
5. Transactions: list → filters → pagination → create/edit/delete.
6. Dashboard & charts.
7. (Optional) Budgets.

End of Frontend Technical Requirements v1.0. These conventions are the direct basis for the automated code-review rules (CLAUDE.md), which enforce the same principles on every pull request.